

Prueba Sumativa N° 1

Programación – 20 de abril de 2024

Nombre		RUT	
Paralelo	F. Regodeceves C1 () C2 () D. Ramírez C3 () M. Moraga C4 () A. Paolini C5 ()		

Antecedentes generales:

Puntaje total de la prueba	1 punto	Puntos para nota aprobatoria	0.6 puntos	Duración de la prueba	3 horas
Resultados de Aprendizaje a evaluar	RA2. Construir programas que satisfagan las especificaciones, verificando el comportamiento esperado de las soluciones implementadas. RA4. Identificar los objetivos y requerimientos de las soluciones TIC. RA5. Desarrollar la solución tecnológica más adecuada en base a las características del problema y los recursos disponibles.				
Fecha de entrega de resultados	11 de mayo de 2024				

Instrucciones:

- 1. Esta evaluación tiene 5 páginas (ruteo por separado) y 4 problemas. Compruebe que dispone de todas las páginas.
- 2. Durante la prueba no se puede utilizar: teléfono móvil, apuntes y/o computador (exceptuando el uso de Spyder). Está prohibido intentar conectarse a internet de cualquier manera. Si es sorprendido obtendrá la calificación mínima. Tampoco puede utilizar dispositivos de almacenamiento externos o cualquier otro dispositivo como relojes inteligentes, audífonos, etc.
- 3. **Lea la prueba completamente DOS veces antes de hacer cualquier pregunta.**
- 4. Una prueba respondida correctamente en un 60%, de acuerdo a las ponderaciones asignadas, corresponde a una nota 4,0.
- 5. La prueba es individual, cualquier sospecha de copia será calificada con la nota mínima y el caso será remitido al comité de ética.
- 6. Recuerde colocar su nombre en una de las primeras 5 línea de código de cada archivo, de lo contrario tendrá un descuento del 25% en cada problema que no lo tenga.
- 7. En su espacio personal no debe haber nada más que hojas de papel en blanco, lápiz, goma y/o calculadora.
- 8. El resto de sus implementos debe guardarlos dentro de su mochila/bolso y ésta debe posicionarse al frente debajo de la pizarra. Si leyó hasta este punto, felicidades, para saber que lo hizo encierre el logo de la Escuela de Ingeniería en esta página.

Acepto las condiciones firmando: _____

Problema 1. Ruteo (20%) (30 minutos) Entrega independiente

Problema 2. OWWO (25%)

En los vastos dominios de la comercialización, donde los comerciantes buscan perpetuar su imperio de transacciones, surge un titánico desafío: la creación de un sistema impositivo para la venerada tienda OWWO, bastión de provisiones y refugio para los viajeros fatigados.

La tienda OWWO, necesita la creación de un programa en Python capaz de procesar los productos que vende, estos están separados en 3 categorías Belleza(B), Limpieza(L) y Comida(C), sin embargo, la dificultad está en que debido al gobierno de la Nación de Elihc, se les debe aplicar un impuesto diferente dependiendo de la categoría Belleza 20%, Limpieza 10% y Comida 15%.

Ejemplo, si compro un producto de Belleza de valor 1000 su precio con el impuesto aplicado será 1200.

Especificaciones del sistema:

- Debe solicitar la categoría según su abreviación (B, L y C):
- Debe Finalizar con la palabra FIN
- Debe funcionar para mayúsculas y minúsculas
- Debe manejar los errores, debe validar que la categoría exista, que los precios sean de valor positivo

Finalmente se solicita lo siguiente:

- a) El impuesto aplicado más alto de la ejecución.
- b) El impuesto aplicado más bajo de la ejecución.
- c) Monto total pagado en impuestos.
- d) Porcentaje de productos de Belleza, Limpieza y Comida en relación al total.
- e) Monto total por pagar (Incluyendo los impuestos).

Ejemplo de ejecución

```
Ingrese categoría del producto, Belleza=B, Limpieza=L, Comida=C: qwe
Ingrese categoría del producto, Belleza=B, Limpieza=L, Comida=C: b
Ingrese precio del producto: 10000
Valor del impuesto a pagar: 2000.0
Valor final por pagar por el producto: 12000.0

Ingrese categoría del producto, Belleza=B, Limpieza=L, Comida=C: L
Ingrese precio del producto: 15000
Valor del impuesto a pagar: 1500.0
Valor final por pagar por el producto: 16500.0

Ingrese categoría del producto, Belleza=B, Limpieza=L, Comida=C: asd
Ingrese categoría del producto, Belleza=B, Limpieza=L, Comida=C: C
Ingrese precio del producto: -1000
Ingrese precio del producto: 20000
Valor del impuesto a pagar: 3000.0
Valor final por pagar por el producto: 23000.0

Ingrese categoría del producto, Belleza=B, Limpieza=L, Comida=C: B
Ingrese precio del producto: 10000
Valor del impuesto a pagar: 2000.0
Valor final por pagar por el producto: 12000.0

Ingrese categoría del producto, Belleza=B, Limpieza=L, Comida=C: fin

a) El impuesto más alto fue de: 3000.0 En un producto de Comida
b) El impuesto más bajo fue de: 1500.0 En un producto de Limpieza
c) El monto total de impuesto es: 8500.0
```

d) El porcentaje de productos de Belleza es: 50%
El porcentaje de productos de Limpieza es: 25%
El porcentaje de productos de Comida es: 25%
e) Total a pagar: 63500.0

Problema 3. Almacenamiento lleno (25%)

Los computadores luego de meses o años de uso comienzan a llenarse de información. Archivos de distintos tipos, como música, videos y otros que corresponden a archivos ocultos o del sistema, que a la larga ralentizan el funcionamiento. Es por esto que un usuario quiere comenzar a liberar espacio en su unidad, con el fin de mejorar el rendimiento.

Para esto debe evaluar qué archivos borrar y qué archivos mantener. Es aquí donde entra tú. Deberás ayudar a definir qué archivos deberá borrar o mantener en base a un análisis de los datos.

Tendrás que crear un programa que dado el archivo llamado **archivos.txt**:

nombre_archivo;tamaño;fecha_creación;fecha_modificación;atributo

donde

nombre_archivo: es el nombre y extensión del archivo separados por un punto.

tamaño: es el tamaño en kilobytes del archivo.

fecha_creación: fecha de creación del archivo en formato MM/DD/YYYY

fecha_modificación: fecha de modificación del archivo en formato MM/DD/YYYY

atributo: puede ser **system**, **read-only** o **hidden**, lo que modificará el modo de uso.

Calcule lo siguiente:

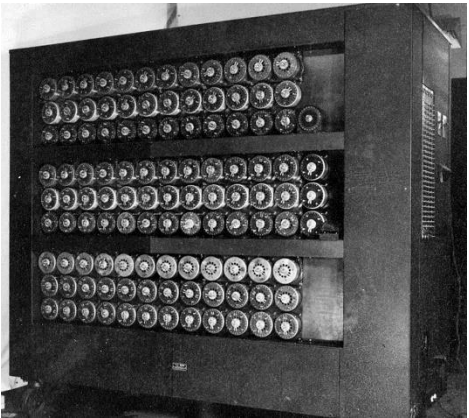
- Porcentaje de archivos corruptos: Un archivo corrupto será el que tenga un año de creación superior al año de modificación. **Para el resto de los cálculos no debe considerar los archivos corruptos.**
- Tamaño de archivos de video en megabytes MB. Considere las extensiones avi, mov y mpeg como archivos de video. Recuerde que 1000 KB es 1 MB.
- Tamaño promedio de los archivos del sistema en KB.

Salida
1) Porcentaje de archivos corruptos: 23.37%
2) Tamaño utilizado por archivos de video: 69400.068 MB
3) Tamaño promedio de archivos del sistema: 504467.36 KB

Problema 4. Código Enigma (30%)

El Código Enigma fue una máquina de cifrado utilizada por las fuerzas armadas alemanas durante la Segunda Guerra Mundial para codificar y descifrar mensajes secretos. Utilizaba una serie de rotores para sustituir letras en el mensaje original, lo que lo hacía extremadamente difícil de descifrar sin el conocimiento de la configuración exacta de la máquina. Este cifrado se consideraba invulnerable en ese momento.

Sin embargo, Alan Turing, un brillante matemático británico, fue una figura clave en la lucha contra el Enigma y en el desarrollo de la tecnología que finalmente llevó a descifrarlo. Turing fue uno de los principales responsables del desarrollo de la máquina "Bombe", un dispositivo



electromecánico utilizado por los británicos para descifrar los mensajes codificados del Enigma. La Bombe realizaba una serie de pruebas para determinar la configuración de los rotores del Enigma y así descifrar los mensajes.

La contribución de Turing fue crucial para la victoria de los Aliados en la Segunda Guerra Mundial, ya que el descifrado de los mensajes del Enigma proporcionó información invaluable sobre los movimientos y planes militares de las fuerzas alemanas. Esta inteligencia permitió a los Aliados anticipar y contrarrestar los movimientos enemigos, lo que, en última instancia, ayudó a cambiar el curso de la guerra.

Con este contexto, el problema consiste en descifrar un mensaje secreto que se encuentra en el archivo código_enigma.txt.

El archivo tiene la siguiente estructura:

- Las líneas impares contienen información del separador a utilizar y de la posición del String a recuperar.
- Las líneas pares contienen el mensaje cifrado.
- El término del archivo está definido por el texto: “endwar”

Consideraciones:

- Las líneas impares siempre utilizan los mismos separadores. Analice archivo para identificar.
- Las líneas pares utilizan el separador indicado por la línea impar directamente anterior.
- El mensaje oculto se va formando con todos los Strings recuperados de las posiciones especificadas por las líneas impares.
- Considere que la revisión se puede realizar con otro archivo que sigue las mismas reglas, pero puede contener otro mensaje codificado.

CODIGO_EJEMPLO.txt	Resultado
SEP: ; ?V=2 PALABRA;HOLA;ESTE;ES SEP:=?V=3 _ = _ = ---- SEP: , ?V=4 123: , ES, 222, XX33XXX, ES, A, A, W3E, A SEP:=?V=3 _ = _ = ---- SEP: ¿ ?V=0 UN¿ 22¿ERROR¿ ¡ SEP:=?V=3 _ = _ = ---- SEP:=?V=7 _ = _ = ----=2=as=EJEMPLO=por=iiii=ooo endwar	ESTE ES UN EJEMPLO

Con el fin de que al revisar su prueba se entienda mejor lo que realizó, utilice nombres de variables que tengan relación con el propósito que tienen. Si va a crear un contador, prefiera nombres como “contador” o “cont” y evite nombres como “c”. Así su profesor no tendrá problemas en interpretar su código. Si leíste esto, escribe mapache al final de la primera página.